
MAZE ROBOT

Dossier Technique — Version STM32F446RE

STM32F446RET6 · HC-SR04 · MPU-6050 · BTS7960B · Flood Fill

Équipe Maze Robot

Carte de développement : NUCLEO-F446RE

Toolchain : STM32CubeIDE · HAL · GCC

Version 1.0 — Mai 2026

Résumé exécutif

Ce dossier décrit l'implémentation complète d'un robot autonome de résolution de labyrinthe basé sur le microcontrôleur **STM32F446RETx** (ARM Cortex-M4 @ 168 MHz). Le système intègre trois capteurs ultrasoniques HC-SR04, une IMU MPU-6050 via I2C, deux drivers moteur BTS7960B, et deux encodeurs en quadrature. L'algorithme de navigation repose sur un **Flood Fill dynamique** (BFS) avec asservissement double (encodeurs + gyroscope).

Table des matières

1	Présentation du Projet	7
1.1	Contexte et Objectifs	7
1.1.1	Objectifs techniques	7
1.2	Comparaison avec la version ESP32	8
2	Design Mécanique	9
2.1	Vue d'ensemble de la structure	9
2.1.1	Caractéristiques mécaniques	9
2.2	Système de transmission différentielle	9
3	Architecture Électrique	10
3.1	Vue d'ensemble	10
3.2	Résumé des composants électroniques	11
3.3	Diviseurs de tension pour HC-SR04	11
4	Configuration STM32CubeMX	12
4.1	Microcontrôleur cible	12
4.2	Configuration de l'horloge (RCC)	12
4.3	Affectation des broches GPIO	14
4.4	Configuration des Timers	14
4.4.1	TIM1 — Génération PWM (contrôle moteurs)	14
4.4.2	TIM2 — Encodeur quadrature (moteur gauche)	15
4.4.3	TIM3 — Encodeur quadrature (moteur droit)	15
4.4.4	TIM5 — Mesure de temps μ s (HC-SR04)	16
4.5	Configuration I2C1 — MPU-6050	16
4.6	Configuration USART2	16

5	Architecture Logicielle	17
5.1	Structure du projet STM32CubeIDE	17
5.2	Modules logiciels recommandés	18
5.3	Diagramme de flux principal	20
6	Drivers Matériels (HAL STM32)	21
6.1	Driver HC-SR04 — Mesure ultrasonique	21
6.1.1	Principe de fonctionnement	21
6.1.2	Code HAL — Mesure distance	21
6.1.3	Instances HC-SR04	22
6.2	Driver BTS7960B — Contrôle moteurs PWM	23
6.2.1	Principe	23
6.2.2	Code HAL — Contrôle PWM	23
6.3	Driver MPU-6050 — Gyroscope I2C	24
6.3.1	Lecture de l'axe Z	24
6.4	Lecture des encodeurs quadrature	25
7	Algorithme de Navigation — Flood Fill	27
7.1	Principe du Flood Fill Dynamique	27
7.2	Représentation de la carte	27
7.3	Implémentation BFS	28
7.4	Mise à jour des parois	28
8	Asservissement et Correction de Trajectoire	30
8.1	Asservissement double	30
8.2	Virage gyroscopique — 3 phases	30
8.3	Paramètres de configuration	31
9	Procédure de Mise en Service et Tests	32
9.1	Ordre de mise en service	32
9.2	Guide de dépannage	33
10	Scripts Linker et Mémoire	34
10.1	Organisation mémoire	34
11	Conclusion	35

11.1 Bilan technique	35
11.2 Perspectives d'amélioration	35

Table des figures

5.1	Diagramme de flux — boucle de navigation principale	20
-----	---	----

Liste des tableaux

1.1	Comparaison STM32F446RE vs ESP32-WROOM-32D	8
3.1	Composants électroniques du Maze Robot STM32	11
4.1	Caractéristiques du STM32F446RETx	12
4.2	Arbre d’horloge — STM32F446RE	13
4.3	Tableau d’affectation des broches — maze_test.ioc	14
4.4	Paramètres TIM1 — PWM moteurs	15
4.5	Paramètres TIM2 — Encodeur gauche	15
4.6	Paramètres TIM3 — Encodeur droit	15
4.7	Paramètres TIM5 — Compteur ps	16
4.8	Paramètres I2C1 — MPU-6050	16
4.9	Paramètres USART2	16
5.1	Organisation modulaire du firmware STM32	18
8.1	Niveaux de correction dans move_forward()	30
8.2	Paramètres critiques — config.h	31
9.1	Procédure de mise en service complète	32
9.2	Tableau de dépannage	33
10.1	Carte mémoire STM32F446RETx	34

Chapitre 1

Présentation du Projet

1.1 Contexte et Objectifs

Le **Maze Robot STM32** est un robot mobile autonome conçu pour explorer et résoudre un labyrinthe inconnu. Il s'appuie sur la carte de développement **NUCLEO-F446RE** embarquant le microcontrôleur **STM32F446RET6**, cœur ARM Cortex-M4 à 168 MHz avec FPU matérielle.

1.1.1 Objectifs techniques

- ✓ Résolution autonome d'un labyrinthe de taille configurable
- ✓ Détection temps réel des obstacles via 3 capteurs HC-SR04 (avant, gauche, droite)
- ✓ Correction de trajectoire double : encodeurs en quadrature + gyroscope MPU-6050
- ✓ Génération PWM haute résolution via Timer TIM1 (168 MHz APB2)
- ✓ Communication debug via USART2 (PA2/PA3) @ 115200 baud
- ✓ Interface de mesure de temps précise via TIM5 (μs)

1.2 Comparaison avec la version ESP32

TABLE 1.1 – Comparaison STM32F446RE vs ESP32-WROOM-32D

Critère	STM32F446RE	ESP32-WROOM-32D
Cœur	ARM Cortex-M4 @ 168 MHz	Xtensa LX6 Dual-Core @ 240 MHz
FPU	Oui (matérielle)	Oui
RAM	128 kB SRAM	520 kB SRAM
Flash	512 kB	4 MB
Timers avancés	TIM1 (centre-aligné, mort-temps)	LEDC (PWM logiciel)
Encodeurs	TIM2 + TIM3 (hardware QEI)	Interruptions logicielles
Capteurs ToF/US	HC-SR04 (GPIO + TIM5 μ s)	VL53L0X I2C (ToF)
Connectivité	USART/SWD	Wi-Fi + Bluetooth
Toolchain	STM32CubeIDE / HAL	Arduino IDE / PlatformIO
Débogage	SWD/JTAG hardware	UART logiciel

Chapitre 2

Design Mécanique

2.1 Vue d'ensemble de la structure

La structure mécanique a été réalisée sous **SolidWorks** en adoptant une géométrie prismatique **décagonale** (10 faces), offrant un volume interne optimisé pour l'électronique tout en fournissant de multiples surfaces planes pour l'intégration des capteurs.

2.1.1 Caractéristiques mécaniques

- **Châssis** : panneaux découpés au laser (MDF ou acrylique), système d'assemblage par tenons et mortaises (finger joint) pour auto-alignement et rigidité structurelle
- **Propulsion** : transmission différentielle avec roues à haute adhérence montées sur axe central
- **Découpes capteurs** : emplacements dédiés sur plusieurs faces pour les capteurs ultrasoniques HC-SR04, permettant une cartographie environnementale à 270°
- **Structure interne** : disposition multi-niveaux avec entretoises et plaques de montage pour organiser les cartes électroniques

2.2 Système de transmission différentielle

Le robot emploie un système de **transmission différentielle** à 2 roues motrices. Les virages sont réalisés par différentiel de vitesse entre la roue gauche et la roue droite, sans mécanisme de direction physique. Ce choix offre une manœuvrabilité maximale (rotation sur place) et une architecture logicielle simple.

Chapitre 3

Architecture Électrique

3.1 Vue d'ensemble

L'architecture électrique est centrée autour du **STM32F446RET6** exploitant son cœur ARM Cortex-M4 haute performance pour gérer en temps réel le traitement des capteurs et le contrôle des moteurs.

3.2 Résumé des composants électroniques

TABLE 3.1 – Composants électroniques du Maze Robot STM32

Système	Composant	Rôle	Interface
Contrôle	STM32F446RETx	Microcontrôleur principal	PWM, GPIO, I2C, UART
Actionnement	2× BTS7960B	Drivers moteur H-Bridge	PWM + GPIO (enable)
Propulsion	2× Moteur DC	Roue gauche + droite	H-Bridge
Odométrie	2× Encodeur QEI	Comptage de déplacements	TIM2, TIM3 (QEI)
Obstacle	3× HC-SR04	Détection avant/-gauche/droite	GPIO + TIM5 (µs)
IMU	MPU-6050	Gyroscope + accéléromètre	I2C1 (PB8/PB9)
Protection	Diviseurs résistifs	Level-shift Echo 5V→3.3V	Passif (1k+2k)
Alimentation	Régulateurs	12V→5V, 5V→3.3V	DC-DC / LDO
Debug	USART2	Liaison série PC	PA2 (TX), PA3 (RX)

3.3 Diviseurs de tension pour HC-SR04



Attention

Les broches Echo du HC-SR04 délivrent un signal à **5 V logique**, incompatible avec les GPIOs du STM32F446 qui tolèrent au maximum **3.3 V**. Un diviseur résistif est indispensable pour chaque signal Echo.

Le diviseur résistif utilisé est composé de $R_1 = 1\text{ k}\Omega$ (série) et $R_2 = 2\text{ k}\Omega$ (vers GND), produisant :

$$V_{\text{GPIO}} = V_{\text{Echo}} \times \frac{R_2}{R_1 + R_2} = 5\text{ V} \times \frac{2}{3} \approx 3.33\text{ V} \leq 3.6\text{ V} \quad (3.1)$$

D'après le schéma du projet, les réseaux utilisent des valeurs de $1\text{ k}\Omega$ sur les lignes Echo avec une référence vers GND, assurant la protection des entrées GPIO.

Chapitre 4

Configuration STM32CubeMX

4.1 Microcontrôleur cible

TABLE 4.1 – Caractéristiques du STM32F446RETx

Paramètre	Valeur
Référence	STM32F446RET6
Famille	STM32F4 (ARM Cortex-M4)
Boîtier	LQFP64
Fréquence max	180 MHz
SRAM	128 kB
Flash	512 kB
Timers avancés	TIM1 (4 canaux, mort-temps)
Timers GP	TIM2, TIM3, TIM5
I2C	I2C1 (mode Fast 400 kHz)
USART	USART2 (asynchrone)
ADC	12 bits, 3 ADC
FPU	Oui (Cortex-M4 FPU)
Carte dev	NUCLEO-F446RE

4.2 Configuration de l’horloge (RCC)

Le fichier `maze_test.ioc` configure le système d’horloge comme suit :

TABLE 4.2 – Arbre d’horloge — STM32F446RE

Paramètre	Valeur
Source PLL	HSE (oscillateur externe 8 MHz)
PLLM	8
PLLN	336
SYSCLK	168 MHz
HCLK (AHB)	168 MHz
APB1 (TIM2, TIM3, TIM5)	42 MHz → Timer clock 84 MHz
APB2 (TIM1)	84 MHz → Timer clock 168 MHz
FLASH latency	5 cycles (FLASH_LATENCY_5)

i Note

Le prescaler de TIM5 est configuré à **167** ($PRE+1 = 168$), ce qui avec $HCLK = 168\text{ MHz}$ donne une résolution de **1 μs** par tick, idéale pour la mesure du temps de vol des capteurs HC-SR04.

4.3 Affectation des broches GPIO

TABLE 4.3 – Tableau d’affectation des broches — maze_test.ioc

Broche	Label	Mode	Description
PA0	—	TIM2_CH1 (Encoder)	Encodeur gauche phase A
PA1	—	TIM2_CH2 (Encoder)	Encodeur gauche phase B
PA2	USART_TX	USART2_TX	Transmission série (debug)
PA3	USART_RX	USART2_RX	Réception série
PA5	LD2 [Green Led]	GPIO_Output	LED verte (onboard)
PA6	—	TIM3_CH1 (Encoder)	Encodeur droit phase A
PA7	—	TIM3_CH2 (Encoder)	Encodeur droit phase B
PA8	—	TIM1_CH1 (PWM)	PWM moteur — canal 1
PA9	—	TIM1_CH2 (PWM)	PWM moteur — canal 2
PA10	—	TIM1_CH3 (PWM)	PWM moteur — canal 3
PA11	—	TIM1_CH4 (PWM)	PWM moteur — canal 4
PA13	TMS	SYS_JTMS-SWDIO	Debug SWD (données)
PA14	TCK	SYS_JTCK-SWCLK	Debug SWD (horloge)
PB0	TRIG_FRONT	GPIO_Output	Trigger HC-SR04 avant
PB1	ECHO_FRONT	GPIO_Input	Echo HC-SR04 avant
PB2	TRIG_LEFT	GPIO_Output	Trigger HC-SR04 gauche
PB3	SWO	SYS_JTDO-SWO	Debug SWO
PB8	—	I2C1_SCL	Horloge I2C (MPU-6050)
PB9	—	I2C1_SDA	Données I2C (MPU-6050)
PB10	ECHO_LEFT	GPIO_Input	Echo HC-SR04 gauche
PB12	TRIG_RIGHT	GPIO_Output	Trigger HC-SR04 droite
PB13	ECHO_RIGHT	GPIO_Input	Echo HC-SR04 droite
PC13	—	GPIO_Input	Bouton utilisateur (B1)
PC14	—	RCC_OSC32_IN	LSE oscillateur 32 kHz
PC15	—	RCC_OSC32_OUT	LSE oscillateur 32 kHz
PH0	—	RCC_OSC_IN	HSE oscillateur 8 MHz
PH1	—	RCC_OSC_OUT	HSE oscillateur 8 MHz

4.4 Configuration des Timers

4.4.1 TIM1 — Génération PWM (contrôle moteurs)

TIM1 est le timer avancé d’APB2 (clock 168 MHz) configuré en mode **PWM Generation** sur 4 canaux simultanés.

TABLE 4.4 – Paramètres TIM1 — PWM moteurs

Paramètre	Valeur
Clock TIM1	168 MHz ($APB2 \times 2$)
Prescaler (PSC)	0 (pas de division)
Auto-Reload (ARR)	9999
Fréquence PWM	$168 \text{ MHz} / (0 + 1) / (9999 + 1) = \mathbf{16.8 \text{ kHz}}$
Résolution PWM	10000 niveaux (0–9999)
Canaux	CH1 (PA8), CH2 (PA9), CH3 (PA10), CH4 (PA11)
Auto-Reload Preload	Activé

Note

La fréquence de 16.8 kHz est supérieure à 20 kHz audible pour la plupart des moteurs DC, évitant les sifflements sonores gênants. L'ARR = 9999 donne **10000 niveaux de résolution PWM**, soit une précision de 0.01%.

4.4.2 TIM2 — Encodeur quadrature (moteur gauche)

TABLE 4.5 – Paramètres TIM2 — Encodeur gauche

Paramètre	Valeur
Mode	Encoder_Interface (TIM_ENCODERMODE_TI12)
Période (ARR)	65535 (compteur 16 bits)
Phase A	PA0 (TIM2_CH1)
Phase B	PA1 (TIM2_CH2)
Auto-Reload Preload	Activé
Comptage	Montant et descendant sur TI1 et TI2

4.4.3 TIM3 — Encodeur quadrature (moteur droit)

TABLE 4.6 – Paramètres TIM3 — Encodeur droit

Paramètre	Valeur
Mode	Encoder_Interface (TIM_ENCODERMODE_TI12)
Phase A	PA6 (TIM3_CH1)
Phase B	PA7 (TIM3_CH2)
Filtre IC1	15 (anti-rebond maximal)
Auto-Reload Preload	Activé

4.4.4 TIM5 — Mesure de temps μs (HC-SR04)

TABLE 4.7 – Paramètres TIM5 — Compteur μs

Paramètre	Valeur
Source clock	Interne ($\text{APB1} \times 2 = 84 \text{ MHz}$)
Prescaler (PSC)	167
Résolution	$84 \text{ MHz} / (167 + 1) = 1 \mu\text{s}/\text{tick}$
Période (ARR)	65535 (débordement après 65.5 ms)
Auto-Reload Preload	Activé

4.5 Configuration I2C1 — MPU-6050

TABLE 4.8 – Paramètres I2C1 — MPU-6050

Paramètre	Valeur
Mode	I2C Fast (400 kHz)
SDA	PB9
SCL	PB8
Adresse MPU-6050	0x68 ($\text{AD0} \rightarrow \text{GND}$) ou 0x69 ($\text{AD0} \rightarrow \text{VCC}$)

4.6 Configuration USART2

TABLE 4.9 – Paramètres USART2

Paramètre	Valeur
Mode	Asynchrone
TX	PA2
RX	PA3
Baud rate (recommandé)	115200 baud
Format	8N1

Chapitre 5

Architecture Logicielle

5.1 Structure du projet STM32CubeIDE

Le projet généré par STM32CubeMX `maze_test` suit la structure standard HAL :

```
maze_test/  
  Core/  
    Inc/  
      main.h  
      stm32f4xx_hal_conf.h  
      stm32f4xx_it.h  
    Src/  
      main.c           ← Point d'entrée + logique navigation  
      stm32f4xx_hal_msp.c ← Init périphériques HAL  
      stm32f4xx_it.c    ← ISR (SysTick, NVIC)  
      syscalls.c  
  Drivers/  
    CMSIS/  
      STM32F4xx_HAL_Driver/  
maze_test.ioc           ← Configuration CubeMX  
STM32F446RETX_FLASH.ld ← Script linker Flash  
STM32F446RETX_RAM.ld   ← Script linker RAM
```

5.2 Modules logiciels recommandés

TABLE 5.1 – Organisation modulaire du firmware STM32

Fichier	Dépendance	Rôle
<code>main.c</code>	HAL	Point d'entrée, boucle principale, Flood Fill
<code>motors.h</code>	TIM1 (PWM)	Init PWM, <code>drive()</code> , <code>stop()</code>
<code>encoders.h</code>	TIM2, TIM3 (QEI)	Lecture compteurs, calcul distance
<code>hcsr04.h</code>	TIM5 (μ s), GPIO	Mesure distance ultrasonique
<code>mpu6050.h</code>	I2C1	Lecture gyroscope, calibration
<code>movement.h</code>	motors + encoders	<code>move_forward()</code> , <code>turn_left/right()</code>
<code>flood_fill.h</code>	—	BFS, <code>updateWalls()</code> , <code>moveToNextCell()</code>
<code>debug.h</code>	USART2	<code>printf</code> redirigé vers USART2

5.3 Diagramme de flux principal

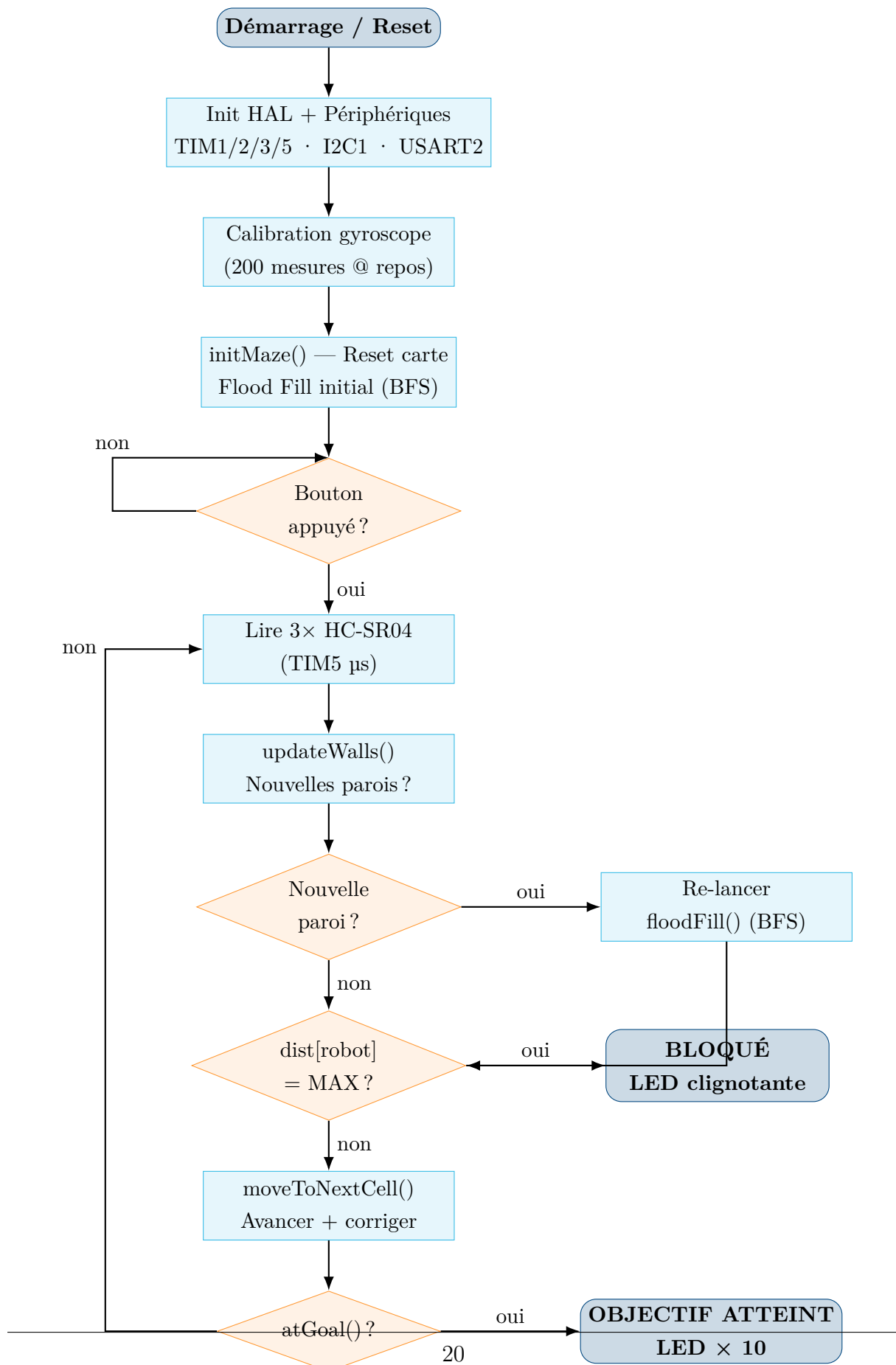


FIGURE 5.1 – Diagramme de flux — boucle de navigation principale

Chapitre 6

Drivers Matériels (HAL STM32)

6.1 Driver HC-SR04 — Mesure ultrasonique

6.1.1 Principe de fonctionnement

1. Envoyer une impulsion TRIG de 10 μ s (niveau haut)
2. Attendre le front montant du signal ECHO
3. Démarrer TIM5 (résolution 1 μ s)
4. Attendre le front descendant de ECHO
5. Arrêter TIM5 — lire le compteur Δt
6. Distance : $d = \Delta t \times 0.0343/2$ [cm]

6.1.2 Code HAL — Mesure distance

```
1  /* hcsr04.h      Mesure distance ultrasonique via TIM5 (1 us/tick) */
2
3  /* R initialise le compteur TIM5 */
4
5  #define SPEED_OF_SOUND_CM_US  0.0343f
6  #define HC_TIMEOUT_US         25000  /* 25 ms max (4 m) */
7
8  typedef struct {
9      GPIO_TypeDef *trig_port;
10     uint16_t      trig_pin;
11     GPIO_TypeDef *echo_port;
12     uint16_t      echo_pin;
13 } HCSR04_t;
14
15 static inline void TIM5_reset(void) {
16     __HAL_TIM_SET_COUNTER(&htim5, 0);
17 }
```

```

16
17 /* Retourne le compteur TIM5 en s */
18 static inline uint32_t TIM5_read_us(void) {
19     return __HAL_TIM_GET_COUNTER(&htim5);
20 }
21
22 float HCSR04_measure_cm(HCSR04_t *sensor) {
23     uint32_t t_start, t_end, elapsed;
24
25     /* --- 1. Impulsion TRIG 10 s --- */
26     HAL_GPIO_WritePin(sensor->trig_port, sensor->trig_pin,
27         GPIO_PIN_SET);
28     TIM5_reset();
29     while (TIM5_read_us() < 10); /* attendre 10 s */
30     HAL_GPIO_WritePin(sensor->trig_port, sensor->trig_pin,
31         GPIO_PIN_RESET);
32
33     /* --- 2. Attendre front montant ECHO --- */
34     TIM5_reset();
35     while (!HAL_GPIO_ReadPin(sensor->echo_port, sensor->echo_pin)
36         ) {
37         if (TIM5_read_us() > HC_TIMEOUT_US) return -1.0f; /*
38             timeout */
39     }
40
41     /* --- 3. Mesurer dur e ECHO haut --- */
42     t_start = TIM5_read_us();
43     while (HAL_GPIO_ReadPin(sensor->echo_port, sensor->echo_pin))
44     {
45         if ((TIM5_read_us() - t_start) > HC_TIMEOUT_US) return
46             -1.0f;
47     }
48     t_end = TIM5_read_us();
49     elapsed = t_end - t_start; /* dur e en s */
50
51     return (float)elapsed * SPEED_OF_SOUND_CM_US / 2.0f;
52 }

```

Listing 6.1 – Driver HC-SR04 — STM32 HAL

6.1.3 Instances HC-SR04

```

1  /* Instances des trois capteurs */
2  HCSR04_t sensor_front = {
3      .trig_port = GPIOB, .trig_pin = GPIO_PIN_0,    /* TRIG_FRONT
4      */
5      .echo_port = GPIOB, .echo_pin = GPIO_PIN_1    /* ECHO_FRONT
6      */
7  };
8  HCSR04_t sensor_left = {
9      .trig_port = GPIOB, .trig_pin = GPIO_PIN_2,    /* TRIG_LEFT
10     */
11     .echo_port = GPIOB, .echo_pin = GPIO_PIN_10    /* ECHO_LEFT
12     */
13 };
14 HCSR04_t sensor_right = {
15     .trig_port = GPIOB, .trig_pin = GPIO_PIN_12,    /* TRIG_RIGHT
16     */
17     .echo_port = GPIOB, .echo_pin = GPIO_PIN_13    /* ECHO_RIGHT
18     */
19 };

```

Listing 6.2 – Initialisation des capteurs HC-SR04

6.2 Driver BTS7960B — Contrôle moteurs PWM

6.2.1 Principe

Chaque BTS7960B est un demi-pont H haute puissance (43 A). Deux puces forment un pont H complet par moteur. Le contrôle se fait via :

- PWM_R / PWM_L : signaux PWM (TIM1) pour la vitesse
- R_EN / L_EN : enable (niveau logique)

6.2.2 Code HAL — Contrôle PWM

```

1  /* D marrage des 4 canaux PWM TIM1 */
2  void motors_init(void) {
3      HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_1); /* PA8 - Moteur G
4      avant */
5      HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_2); /* PA9 - Moteur G
6      arri re */
7      HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_3); /* PA10 - Moteur D
8      avant */
9  }

```



```

6     HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_4); /* PA11 - Moteur D
        arriere */
7 }
8
9  /* speed : 0      9999 (ARR = 9999) */
10 void motor_set_speed(int16_t speed_left, int16_t speed_right) {
11     if (speed_left >= 0) {
12         __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, speed_left);
13         __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_2, 0);
14     } else {
15         __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, 0);
16         __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_2, -speed_left)
            ;
17     }
18     if (speed_right >= 0) {
19         __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_3, speed_right)
            ;
20         __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_4, 0);
21     } else {
22         __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_3, 0);
23         __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_4, -speed_right
            );
24     }
25 }
26
27 void motors_stop(void) {
28     motor_set_speed(0, 0);
29 }

```

Listing 6.3 – Contrôle moteurs via TIM1 PWM

6.3 Driver MPU-6050 — Gyroscope I2C

6.3.1 Lecture de l'axe Z

```

1 #define MPU6050_ADDR      (0x68 << 1)
2 #define MPU6050_PWR_MGMT_1 0x6B
3 #define MPU6050_GYRO_ZOUT_H 0x47
4 #define GYRO_SCALE        131.0f /* LSB / deg/s ( 250 deg/s)
        */
5
6 void MPU6050_init(void) {

```

```

7     uint8_t data = 0x00; /* Wake up (clear sleep bit) */
8     HAL_I2C_Mem_Write(&hi2c1, MPU6050_ADDR, MPU6050_PWR_MGMT_1,
9                       1, &data, 1, 100);
10 }
11
12 float MPU6050_read_gyro_z(void) {
13     uint8_t buf[2];
14     HAL_I2C_Mem_Read(&hi2c1, MPU6050_ADDR, MPU6050_GYRO_ZOUT_H,
15                     1, buf, 2, 100);
16     int16_t raw = (int16_t)((buf[0] << 8) | buf[1]);
17     return (float)raw / GYRO_SCALE; /* deg/s */
18 }
19
20 /* Calibration : moyenne de 200 chantillons au repos */
21 float gyro_bias_z = 0.0f;
22 void MPU6050_calibrate(void) {
23     float sum = 0.0f;
24     for (int i = 0; i < 200; i++) {
25         sum += MPU6050_read_gyro_z();
26         HAL_Delay(5);
27     }
28     gyro_bias_z = sum / 200.0f;
29 }

```

Listing 6.4 – Lecture MPU-6050 via I2C1

6.4 Lecture des encodeurs quadrature

```

1 /* Lecture et remise z ro des compteurs encodeurs */
2 typedef struct {
3     int32_t left;
4     int32_t right;
5 } EncoderCounts_t;
6
7 EncoderCounts_t encoders_read_reset(void) {
8     EncoderCounts_t counts;
9     /* Lire les compteurs 16 bits avec gestion du d bordement */
10    counts.left = (int16_t)__HAL_TIM_GET_COUNTER(&htim2);
11    counts.right = (int16_t)__HAL_TIM_GET_COUNTER(&htim3);
12    /* Remettre z ro */
13    __HAL_TIM_SET_COUNTER(&htim2, 0);
14    __HAL_TIM_SET_COUNTER(&htim3, 0);

```

```
15     return counts;  
16 }
```

Listing 6.5 – Lecture encodeurs TIM2/TIM3

Chapitre 7

Algorithme de Navigation — Flood Fill

7.1 Principe du Flood Fill Dynamique

L'algorithme BFS (*Breadth-First Search*) attribue à chaque cellule une **distance** représentant le nombre minimal de pas pour atteindre l'objectif (cellule objectif = distance 0). Le robot se déplace toujours vers le voisin accessible de plus faible distance. Lorsqu'une nouvelle paroi est découverte, le BFS complet est relancé instantanément.

7.2 Représentation de la carte

```
1 #define MAZE_SIZE      5          /* Grille 5x5 cellules */
2 #define GOAL_X         2
3 #define GOAL_Y         2
4 #define DIST_MAX       255
5
6 /* Bitmasks des parois (par cellule) */
7 #define WALL_N  0x01
8 #define WALL_E  0x02
9 #define WALL_S  0x04
10 #define WALL_W  0x08
11
12 uint8_t walls[MAZE_SIZE][MAZE_SIZE]; /* Parois connues */
13 uint8_t dist [MAZE_SIZE][MAZE_SIZE]; /* Distances BFS */
14
15 int robotX = 0, robotY = 0;          /* Position virtuelle */
16 int robotHeading = 0;                /* 0=N 1=E 2=S 3=W */
17
18 const int DIR_DX[4] = { 0, 1, 0, -1}; /* N E S W */
```

```
19 const int DIR_DY[4] = { 1, 0, -1, 0};
```

Listing 7.1 – Structures de données — Flood Fill

7.3 Implémentation BFS

```
1 typedef struct { uint8_t x, y; } QCell;
2 static QCell _bfsQ[MAZE_SIZE * MAZE_SIZE];
3
4 void floodFill(void) {
5     memset(dist, DIST_MAX, sizeof(dist));
6     int head = 0, tail = 0;
7
8     dist[GOAL_Y][GOAL_X] = 0;
9     _bfsQ[tail++] = (QCell){GOAL_X, GOAL_Y};
10
11     while (head < tail) {
12         QCell cur = _bfsQ[head++];
13         uint8_t nd = dist[cur.y][cur.x] + 1;
14         for (int d = 0; d < 4; d++) {
15             if (walls[cur.y][cur.x] & (1 << d)) continue; /*
16                 paroi */
17             int nx = cur.x + DIR_DX[d];
18             int ny = cur.y + DIR_DY[d];
19             if (nx < 0 || nx >= MAZE_SIZE) continue;
20             if (ny < 0 || ny >= MAZE_SIZE) continue;
21             if (dist[ny][nx] <= nd) continue;
22             dist[ny][nx] = nd;
23             _bfsQ[tail++] = (QCell){nx, ny};
24         }
25     }
```

Listing 7.2 – floodFill() — BFS depuis l'objectif

7.4 Mise à jour des parois

```
1 #define WALL_DISTANCE_CM 20.0f /* Seuil de d t e c t i o n : 20 cm
2     */
3 int updateWalls(void) {
4     int new_wall = 0;
```

```

5
6   float df = HCSR04_measure_cm(&sensor_front);
7   float dl = HCSR04_measure_cm(&sensor_left);
8   float dr = HCSR04_measure_cm(&sensor_right);
9
10  /* Convertir directions relatives      absolues selon cap
11     robot */
12  int dir_front = robotHeading;
13  int dir_left  = (robotHeading + 3) & 3;
14  int dir_right = (robotHeading + 1) & 3;
15
16  /* Enregistrer les parois sym triquement */
17  if (df > 0 && df < WALL_DISTANCE_CM) {
18      if (!(walls[robotY][robotX] & (1 << dir_front))) {
19          walls[robotY][robotX] |= (1 << dir_front);
20          /* Paroi c t voisin aussi */
21          int nx = robotX + DIR_DX[dir_front];
22          int ny = robotY + DIR_DY[dir_front];
23          if (nx >= 0 && nx < MAZE_SIZE && ny >= 0 && ny <
24              MAZE_SIZE)
25              walls[ny][nx] |= (1 << ((dir_front + 2) & 3));
26          new_wall = 1;
27      }
28  }
29  /* R p ter pour gauche et droite... */
30  return new_wall;
31 }

```

Listing 7.3 – updateWalls() — intégration capteurs HC-SR04

Chapitre 8

Asservissement et Correction de Trajectoire

8.1 Asservissement double

La fonction `move_forward()` implémente un asservissement à deux niveaux :

TABLE 8.1 – Niveaux de correction dans `move_forward()`

Source	Variable	Calcul et rôle
Encodeurs	<code>enc_corr</code>	$(e_L - e_R) \times K_{enc}$, borné à ± 40 Corrige le décalage de vitesse instantané
Gyroscope	<code>gyro_corr</code>	$\int \omega_z dt \times K_{gyro}$, borné à ± 25 Corrige la dérive angulaire accumulée
Commande finale		<code>motor_set_speed(speed + corr, speed - corr - MOTOR_BIAS)</code>

$$\text{correction} = K_{enc}(e_L - e_R) + K_{gyro} \int_{t_0}^t \omega_z(\tau) d\tau \quad (8.1)$$

8.2 Virage gyroscopique — 3 phases

1. **Phase rapide** : vitesse `SPEED_TURN` jusqu'à $\theta_{cible} - 12^\circ$
2. **Phase lente** : vitesse réduite (PWM 110) sur les 12° restants
3. **Arrêt + mesure** : frein 80 PWM pendant 55 ms, puis intégration gyroscope pendant 1 s pour mesurer l'angle réel

8.3 Paramètres de configuration

TABLE 8.2 – Paramètres critiques — config.h

Paramètre	Valeur	Unité	Description
MAZE_SIZE	5	cellules	Grille $n \times n$
PULSES_PER_CELL	à calibrer	pulses	Encodeur / cellule (400 mm)
PULSES_90	à calibrer	pulses	Pulses pour virage 90° exact
SPEED_NORMAL	6000	/9999 PWM	Vitesse de croisière
SPEED_TURN	5000	/9999 PWM	Vitesse en virage
MOTOR_BIAS	à mesurer	/9999 PWM	Correction dérive gauche/droite
WALL_DISTANCE_CM	20.0	cm	Seuil HC-SR04 détection paroi
GYRO_SCALE	131.0	LSB/°/s	Sensibilité MPU-6050 ($\pm 250^\circ/\text{s}$)
TURN_SLOW_DEG	12	degrés	Début phase lente du virage
BRAKE_PWM	3000	/9999 PWM	Puissance frein
BRAKE_MS	55	ms	Durée du frein

Chapitre 9

Procédure de Mise en Service et Tests

9.1 Ordre de mise en service



Attention

Respecter impérativement l'ordre de mise en service ci-dessous. **PULSES_PER_CELL** et **PULSES_90** doivent être recalibrés à chaque changement de surface ou de batterie.

TABLE 9.1 – Procédure de mise en service complète

Étape	Test	Procédure et vérification
1	TEST_MOTORS	Activer chaque moteur 1 s. Si une roue tourne à l'envers, inverser ses fils sur le BTS7960B
2	TEST_ENCODERS	Afficher compteurs L/R. Tourner chaque roue à la main : les compteurs doivent augmenter
3	TEST_SENSORS	Afficher distances et parois toutes les 200 ms. Placer un obstacle et vérifier la détection
4	TEST_MOVE	Avancer d'une cellule. Comparer distance réelle à 400 mm. Ajuster PULSES_PER_CELL
5	TEST_SPIN	Effectuer $4 \times 90^\circ$. Ajuster PULSES_90 jusqu'au retour au cap initial ($\pm 2^\circ$)
6	TEST_SQUARE	Carré de test : $4 \times$ (avancer + tourner gauche). Le robot doit revenir à son point de départ
7	NAVIGATION	Tous les flags à 0 $\rightarrow$ Mode labyrinthe $\rightarrow$ Appuyer sur le bouton

9.2 Guide de dépannage

TABLE 9.2 – Tableau de dépannage

Symptôme	Cause probable		Solution
Robot roule de travers	Biais moteurs		Ajuster <code>MOTOR_BIAS</code>
Ne traverse pas 400 mm	<code>PULSES_PER_CELL</code>	incorrect	Re-exécuter <code>TEST_MOVE</code>
Virage $\neq 90^\circ$	<code>PULSES_90</code> incorrect		Re-exécuter <code>TEST_SPIN</code>
HC-SR04 toujours 0	TIM5 non démarré		Vérifier <code>HAL_TIM_Base_Start</code>
HC-SR04 instable	Pas de diviseur R sur Echo		Ajouter pont résistif 1k+2k
MPU-6050 KO	Mauvaise adresse I2C		Vérifier état broche AD0
Gyro tourne à l'envers	Signe gyro incorrect		Inverser <code>GYRO_SIGN</code>
Robot BLOQUÉ	Paroi non détectée		Diminuer <code>WALL_DISTANCE_CM</code>
Encodeurs ne comptent pas	Filtre IC trop fort		Vérifier <code>IC1Filter</code> TIM3

Chapitre 10

Scripts Linker et Mémoire

10.1 Organisation mémoire

TABLE 10.1 – Carte mémoire STM32F446RETx

Région	Adresse	Taille	Contenu
FLASH	0x08000000	512 kB	Code + données constantes
RAM	0x20000000	128 kB	Variables + pile + tas
CCMRAM	—	—	Non disponible (F446)
Stack	RAM haute	0x400 (1 kB)	Défini dans linker
Heap	RAM	0x200 (512 B)	Défini dans linker

Le projet fournit deux scripts linker :

- STM32F446RETX_FLASH.ld — exécution depuis la Flash (production)
- STM32F446RETX_RAM.ld — exécution depuis la RAM (debug rapide)

Chapitre 11

Conclusion

11.1 Bilan technique

Le **Maze Robot STM32F446RE** constitue une implémentation complète et robuste d'un solveur de labyrinthe autonome sur une plateforme embarquée professionnelle.

- **Performance temps-réel** : le cœur Cortex-M4 @ 168 MHz avec FPU garantit un traitement fluide des boucles de contrôle et de l'algorithme BFS
- **Précision odométrique** : les encodeurs quadrature hardware (TIM2/TIM3) offrent une mesure de déplacement fiable sans charge CPU
- **PWM haute résolution** : TIM1 @ 16.8 kHz avec 10000 niveaux permet un contrôle moteur précis et sans bruit audible
- **Mesure μ s précise** : TIM5 avec résolution 1 μ s assure une mesure fiable du temps de vol HC-SR04
- **Architecture HAL** : la couche d'abstraction STM32Cube HAL facilite la portabilité vers d'autres MCU STM32
- **Débogage SWD** : la connexion SWD/SWO native de la NUCLEO-F446RE permet le débogage temps-réel et le profilage de performances

11.2 Perspectives d'amélioration

- Ajout d'un filtre de Kalman 1-D pour le lissage des mesures HC-SR04
- Implémentation du mode Speed Run (enchaînement de cellules droites)
- Interface de monitoring via USART2 + terminal série (Putty/STM32CubeMonitor)
- Migration vers FreeRTOS pour une architecture multi-tâches (capteurs / navigation / debug)
- Remplacement des HC-SR04 par des capteurs VL53L0X I2C pour plus de précision

Bibliographie

- [1] STMicroelectronics, *STM32F446xC/E Reference Manual (RM0390)*, Rev 6, 2019. https://www.st.com/resource/en/reference_manual/rm0390-stm32f446xx-advanced-armbased-32bit-mcus-stmicroelectronics.pdf
- [2] STMicroelectronics, *STM32F446RE Datasheet (DS10693)*, Rev 8, 2021.
- [3] STMicroelectronics, *STM32CubeF4 HAL and Low-Layer Drivers User Manual (UM1725)*, 2023.
- [4] Elecfreaks, *HC-SR04 Ultrasonic Ranging Module Datasheet*, 2013.
- [5] InvenSense, *MPU-6000/MPU-6050 Product Specification Revision 3.4*, 2013.
- [6] Infineon Technologies, *BTS7960B Industrial H-Bridge Datasheet*, Rev 2.0, 2014.
- [7] Micromouse Online, *Flood Fill Algorithm for Maze Solving*, <https://www.micromouseonline.com>.
- [8] A. Benkridis & M. Moujbeni, *Dossier Technique Maze Robot — ESP32 Flood Fill Version 3.0*, Avril 2026.